DAY 28 | HANDS-ON ACTIVITIES

LLM Security and Guardrails: Problem Statements

Seven defensive problems. Only test systems you own or are authorised to test. The vulnerable bot in the notebook is simulated for learning.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Threat Model

Level: Warm-up | **Concepts:** OWASP LLM risks

For a support bot that reads emails and can issue refunds, list the top five risks, an example of each, and one control per risk.

Sample run / expected result

```
Indirect injection via email -> scan and tag email text as data
Excessive agency -> refunds above 2,000 need approval
```

Hints

- Think about inputs, data sources and actions.

Problem 2: Spot the Injection

Level: Easy | **Concepts:** pattern detection

Write a scanner that scores text for injection signals ("ignore previous instructions", "system prompt", "you are now") and run it on 10 attacks and 10 benign messages.

Sample run / expected result

```
Block rate: 9/10
False alarms: 1/10
```

Hints

- Measure both sides: missed attacks and wrongly blocked users.

Problem 3: Defend the RAG Bot

Level: Medium | **Concepts:** context tagging, quarantine

Use the notebook's vulnerable bot. Add context scanning that quarantines poisoned documents, wrap the rest in tags, and show the leak no longer happens.

Sample run / expected result

```
Before: leaked STAFF code
After: quarantined 1 document, safe answer
```

Hints

- Removing the secret from the prompt is the strongest fix.

Problem 4: PII Redaction

Level: Easy | **Concepts:** output filtering

Redact emails, phone numbers (Indian and Japanese formats) and card numbers from model outputs. Test on 6 sample outputs.

Sample run / expected result

```
"Call me on +91 98450 11111" -> "Call me on [phone removed]"
```

Hints

- Test that normal numbers like order totals are not removed.

Problem 5: Approval Gate

Level: Medium | **Concepts:** human in the loop

Classify actions as low, medium or high risk and route them: auto-run, review queue or blocked pending approval. Log every decision.

Sample run / expected result

```
send_summary -> auto
email_customer -> review
issue_refund(5000) -> approval required
```

Hints

- Default unknown actions to the highest risk.

Problem 6: Counterfactual Fairness

Level: Challenging | **Concepts:** bias testing

Send the same pre-screening request with 6 different names and cities. Compare decisions and explanations. Report any differences.

Sample run / expected result

```
Decisions identical for 6/6 variants (offline rule)
Online: check wording differences too
```

Hints

- Change only one attribute at a time.

Problem 7: Red-team Suite

Level: Stretch | **Concepts:** regression tests for safety

Turn your attacks and benign prompts into PyTest tests that fail if block rate drops below 90% or false alarms rise above 10%.

Sample run / expected result

```
4 passed: block_rate=0.9, false_alarm_rate=0.1
```

Hints

- Safety tests should run on every change, like unit tests.

Anzen (安全): safety